

Java オブジェクト指向学習用可視化アプリ

Python / PyQt6 を使った簡易実装・評価と今後の課題

Hiroki Funashima

Sunday, October 4, 2026

概要

本書は、Java のオブジェクト指向概念とデザインパターンを学ぶための可視化アプリを、なにかの研究として何かの研究としてつかえないかなーという計画とパイロット実装をまとめた独立文書である。研究対象を、参照と同一性の基礎教材、および Strategy・State・Observer の 3 パターンに限定する。アプリ本体は Python・PyQt6 で実装し、同梱 Java 教材の実行記録を読み取って、参照、実オブジェクト型、呼び出し、状態変化、ソース位置を提示する。研究評価では、人の学習効果を測定せず、表示モデルと Java の観測状態との整合性、正常実装と契約違反実装の区別、記録の再現性を扱う。実装済みパイロットは 26 シナリオを Java で実行し、86 チェックポイントで観測状態が一致した。正常 13 件と比較 13 件は事前に定義した契約に対して所期の判定となった。ただし誤実装は 4 種類であり、この結果は自作・開発用コーパスにおける動作確認である。未知実装への一般化、教材追加時の保守性の優位、教育効果は未検証である。23 パターンへの網羅的拡張は必須成果から外し、学生が独立した題材と評価を追加することを卒業研究の中心に据える。

目次

1	結論と研究の位置付け	3
2	検討の理由と優先順位	4
3	背景知識と関連研究	5
4	Python 実装の意義をどう扱うか	6
5	研究課題と到達目標	7
6	実装したパイロットの構成	8
7	観測モデルと判定の意味	10
8	GUI の操作と確認範囲	12
9	実施したパイロット評価	13
10	もし卒業研究としてやるとした場合に、追加すべき評価	15
11	今後の課題と検討事項	16
12	たとえば、12 週間の実施例と判断基準	17
13	提供物と学生自身の研究成果の境界	17
14	妥当性と未実装事項	18
15	まとめ	18
付録 A	起動と評価の手順	18

1 結論と研究の位置付け

1.1 研究としての適切な範囲

本研究は、対象を限定すればたとえば、卒業研究として取り組みやすいという結論になった。推奨する題目は次のとおりである。

Java の参照とオブジェクト相互作用を対象とする学習用可視化アプリの試作と実行整合性評価

パターン名を題目に含める場合には、「Strategy・State・Observer を対象とした」と対象範囲を明記する。全 23 パターンの実装、汎用 Java デバッガ、形式的な意味保存証明、大規模な保守性比較を一度に達成する計画にはしない。

たとえば、卒業研究に限っていえば、重要なのは、学生が問題設定、設計判断、実装の原理、実験、限界を説明できることである。本案はその観点からの提案であり、コースでの授業、特に情報実習 2 や 3 などの正式な評価基準を代替するものではない。担当学生の経験、残り期間、教員が提供する基盤の範囲に応じて調整する必要があるかもしれない。

1.2 開発目的と実証する主張

目的は、初学者が Java のプログラムと実行中のオブジェクト関係を対応付けて探索できる環境を提供することである。研究上の主張は、「観測対象の実行状態を正確に再構成できる」「定義した課題の違反を根拠付きで示せる」に限定する。

前提として、

- 人を対象にした学習実験は必須としない。
- 理解度、学習時間、認知負荷、使いやすさ、開発者の主観的可読性は評価しない。

可視化の正しさを確認できても、学習成果が向上したことにはならない。GUI の自動操作試験や開発者による画面確認は、被験者実験とは別の実装検証である。

2 検討の理由と優先順位

2.1 作業量を増やす要因

23 種類すべてを扱うと、Java 教材だけでなく、役割注釈、説明、正常例、誤実装、入力、期待値、図の配置、回帰試験が種類ごとに必要になる。さらに任意の Java コードを受け付けると、構文解析、型情報、計測、例外、外部ライブラリ、実行制御などが追加される。高専卒研の限られた期間では、実装の拡張を優先した結果、研究としての評価と考察が不足するおそれがある。

また、単に「23 種類を作った」ことは教材の量を示すが、表示の正確性や設計の一般性を示すとは限らない。対象を絞っても、何を観測し、どこで破綻するかを調べれば、まとまりのある研究になる。

2.2 必須・選択・対象外

優先度	内容	完了を判断する基準
必須	参照の基礎、Strategy、State、Observer	各教材で正常例と課題違反例を説明し、再生できる。
必須	実行記録・状態再構成・状態照合	Java の直接観測と表示モデルの対応を確認できる。
必須	根拠付きの契約判定	判定の対象と根拠イベントを追跡できる。
必須	再現スクリプトと結果表	他者が同じ条件で結果を再生できる。
卒研で追加	独立した題材・誤実装・評価入力	パイロット作成に使っていない例で限界を確認する。
選択	教材追加時の変更量の調査	変更前後の差分と変更理由を記録する。
選択	Composite または Decorator を 1 種類追加	既存モデルで表せることと追加実装が必要な点を区別する。
対象外	23 種類の完全対応、任意コード解析、並行処理	発展課題として文書化する。
対象外	学習効果、Python の一般的優越性	本研究の結果から結論しない。

基本機能が安定しない段階では新パターンを増やさない。締切が短い場合には、基礎教材と Strategy・Observer に絞る代わりに、独立評価例と評価の説明を維持する。

3 背景知識と関連研究

3.1 Java の意味と教材の関係

Java では、参照型の変数が持つ値はオブジェクトへの参照であり、変数自体がオブジェクトそのものではない [2]。例えば二つの変数が同じ Box を参照すれば、一方を通じたフィールド更新を他方からも観測できる。表示では変数を保持する場所と参照先の実体を分ける。

インターフェース型の参照から呼び出す実装は、実行時の対象に依存する場合がある。メソッド呼び出しの動的な探索は Java 言語仕様に定められている [3]。パイロットでは Strategy の Price と実体の Regular/Discount を使って、宣言型、実オブジェクト型、呼び出し先を対応付ける。ただしオーバーロード解決や Java 全体の型検査を実装するものではない。

GoF のデザインパターンは、再利用可能なオブジェクト指向設計の記述である [1]。パターン採用の価値は具体的な要求と変更に依存するため、クラス数が増えたことだけを改善とは扱わない。教材では委譲や通知による間接化の利点と、要素数・追跡箇所が増える代償の双方を説明する。

3.2 可視化研究との違い

既存研究である Jeliot 3 は初学者向けにデータと制御の流れを可視化する [4]。JavaWiz は実行記録に基づくグラフィカルデバッガであり、ヒープ、スタック、実行履歴等の表示を備える [5]。実行記録へ解説や問いを付ける Pedagogical Execution Traces の構想も公開されている [6]。したがって、実行を図示することや説明を付けること自体を新規としない。

Python Tutor の Java 関連実装 `java_visualize` では、表示フロントエンドを再利用し、Java 用バックエンドへ置き換える構成が説明されている [7]。この先行例は Python・PyQt6 と同一構成ではないが、教育対象言語と表示基盤の分離には先行例があることを示す。

本研究の卒研としての焦点は、少数の課題に対して、(1) 参照・呼び出し・状態を結び付ける、(2) 表示の整合性と教材契約を分ける、(3) 誤実装の根拠を示す、という仕組みを実装し、再現可能に評価することである。既存研究にない機能であることは未確定であり、学術的な新規性を主張する場合は本文と実装をさらに比較する。

4 Python 実装の意義をどう扱うか

4.1 実装言語と教育対象言語を分ける理由

Python は、GUI、JSON 処理、評価スクリプトを一つの言語で扱うために採用する。PyQt6 は Qt の Python バインディングであり [9]、描画は Qt の Graphics View 系の機能を利用する [10]。Java の計算は Java で実行し、Python 側では記録から状態を再構成する。

この分担により、Python で Java 意味論を全面的に作り直す必要を避け、表示と評価のコードを Python 側へ集約できる。ただし全ファイルが Python になるわけではなく、Java 教材と計測補助コードは残る。JDK を使わない再生モードでは、事前に Java で取得した記録を利用する。

4.2 言語選択だけを新規性にしない

「Java の教材を Python で実装した」ことだけでは、研究上の独立した新規性は弱い。「Python だから学生や保守担当者が理解しやすい」という主張にも、本研究には人の理解を測る証拠がない。Python 採用は実装条件として説明し、一般的な言語の優劣は結論しない。

卒研で扱える貢献候補は、同じ Python 基盤の中で説明と役割情報を教材データへ分離し、ある変更がどのファイルまで波及したかを記録することである。例えば「説明文の変更は JSON だけで完了した」「新教材には Java と契約関数の変更が必要だった」と具体的に示す。コード行数だけで保守容易性を断定しない。

4.3 現時点で実証していないこと

パイロットは教材解説・役割・宣言型注釈を JSON に分離したが、教材 ID、契約、Java 実行分岐はコードに記述されている。新教材の追加はデータだけでは完結しない。従って「コアを変更せず 23 種類に拡張可能」「Python 構成が Java 構成より保守しやすい」とは主張しない。別言語で同等の GUI を作り直す比較は、今回の卒研では必須としない。

5 研究課題と到達目標

以下にこのテーマの研究上の問い (Research Question, RQ) の候補を挙げる。

番号	問い	調べる対象
RQ1	定義した観測点で、Python の再構成状態は Java の実状態と一致するか。	チェックポイントの型、フィールド、参照 ID、参照共有。
RQ2	指定した教材契約の違反を、イベント根拠とともに示せるか。	正常例、意図した誤実装、記録欠損、根拠 ID。
RQ3	独立した題材の追加にどの変更が必要か。	変更箇所、観測仕様の不足、契約の作り直し。

RQ1 と RQ2 を必須とし、RQ3 は題材追加の実務記録として小さく実施する。機能量よりも、観測範囲と結果の意味を説明できることを完了条件にする。

もし、万が一卒業研究になったとしたら、学生の最低到達目標は、1 教材について、Java の代入・呼び出し、出力イベント、Python の状態更新、図の変化、契約判定を順に追って説明できることである。さらに、正しく動く例だけでなく、何が不足すると判定できないかを説明する。

6 実装したパイロットの構成

6.1 処理の流れ

同梱 Java プログラムを単一ソースファイルモードで実行し、JSON Lines のイベントを出力する。Java の単一ソースファイル実行は標準の `java` コマンドが提供する機能である [11]。Python はイベントを検証して状態を再構成し、PyQt6 で表示する。Java を起動しない場合は同梱記録を読む。

ファイル・領域	実装した責務
<code>java/PatternPilot.java</code>	4 教材、手動計測、反射による実オブジェクト観測、JSON 出力。
<code>pattern_lab/core.py</code>	入力検証、状態再構成、契約判定、根拠、保存。Qt へ依存しない。
<code>pattern_lab/runner.py</code>	同梱 Java 起動、入力制限、タイムアウト、ソースハッシュ付与。
<code>pattern_lab/gui.py</code>	図、コード、履歴、説明、検証結果、比較、前進・後退。
<code>lessons/*.json</code>	日本語解説、問い、役割と宣言型の注釈。
<code>traces/*.jsonl</code>	正常 4 件・比較 4 件の Java 実行記録。
<code>evaluate.py</code>	境界入力を含む Java 実行と、CSV / JSON への結果出力。
<code>tests/</code>	コアと GUI の自動試験。

複数モジュールに分割したが、パイロット段階では高度なプラグイン機構や契約記述言語を導入しない。学生が全体を把握できる小さな構成を優先する。

6.2 参照と同一性

`References` が `a` と `b` を保持する。正常例では `b = a` とし、`b.value` へ値を代入する。比較例では `b = new Box()` とする。この比較例は Java の文法や型規則に違反しないが、「`a` と `b` が同じ `Box` を参照する」という課題に違反する。

GUI 上では保持元と参照先を別の箱として表示する。入力 0 では両方の値が同じでも、同一性条件で違いを区別する。単に最終的な数値が一致しただけでは同じ振る舞いと結論しないことを示す教材である。

6.3 Strategy

`Checkout` は `Price` 型のフィールドを持つ。最初は `Regular`、次に `Discount` へ差し替える。通常料金は入力値、割引料金は Java 整数演算で入力値の 8 割とする。入力を 0~10000 へ制限し、今回の計算で整数オーバーフローを生じない範囲にする。

比較例では、選択済みの `price` を無視して最初の `Regular` を呼ぶ。入力 0 では結果の数値が同じになるが、呼び出し先の列が異なる。従って出力値だけでは見えない違いを、参照と呼び出しの観測で示

せる。ただしこれは今回の契約に対する違反であり、任意の Strategy 実装を判定する規則ではない。

6.4 State

Gate は Locked から Open へ遷移し、次の要求で通過回数を増やして Locked へ戻る。比較例は最初の状態変更を省略する。処理を担当する実体と、Context が保持する状態の変化を追跡する。

Strategy と構造が似ていることを示しながら、State では要求と状態遷移の關係に注目する。構造だけで両パターンを自動分類する機能は持たない。State 教材は固定した 2 回の要求を使い、GUI の数値入力は無効にする。

6.5 Observer

Subject に 2 つの Display を登録し、最初の通知を行う。その後に一方を解除し、残る一方へ再通知する。比較例は各通知時に登録リストの最後の相手を呼ばない。

前提は、同期通知、重複登録なし、通知中の登録変更なし、コールバック例外なしである。この前提の下で、開始時の登録先へちょうど一度ずつ通知する契約を検証する。通知順序自体は契約に含めない。これは Observer 一般のすべての実装へ強制する条件ではない。

7 観測モデルと判定の意味

7.1 イベントと状態

実行記録を $T = (e_1, \dots, e_n)$ 、初期状態を S_0 、イベント適用関数を F とすれば、

$$S_{k+1} = F(S_k, e_{k+1})$$

によって状態を再構成する。 e_k は第 k イベント、 S_k はその時点で観測しているオブジェクトの表である。JVM 全体の状態ではない。

主なイベントは生成登録 `new`、フィールド更新 `set`、呼び出し `call`、復帰 `return`、照合点 `checkpoint` である。参照は `{"ref": "object_id"}` に相当する JSON オブジェクトとして表す。実オブジェクト型は Java 側から取得し、宣言型は教材注釈として別に与える。

後退操作は、保存済みの記録を先頭から指定位置まで再生する。JVM を逆実行する機能ではない。処理量は記録長に概ね比例するため、長大な記録向けの効率化は今後の課題である。

7.2 観測整合性と教材契約を分ける

表示用の再構成状態を S_k 、実 Java オブジェクトから直接取得した観測状態を J_k とすると、対象チェックポイントで

$$S_k = J_k$$

を照合する。両者は同じオブジェクト ID とフィールド形式で表される。`checkpoint` の内容を状態の更新には使わず、`new` と `set` から構成した状態との比較だけに用いる。

一方、教材契約は、例えば「交換後に Discount へ委譲する」「解錠後は Open である」「全登録先へ通知する」という要求である。誤実装をそのとおり再生した場合、観測整合性は PASS、教材契約は FAIL になる。両判定を同じものとして扱わない。

7.3 独立性の範囲

Java の照合用スナップショットは、反射で実オブジェクトのフィールドを読み取り、出力済みイベントを参照しない。このため、フィールド更新の計測を取り忘れた場合を検出できる。一方、生成時と照合時で ID・値変換の補助処理を共有するので、完全に独立した二実装ではない。型変換器に同じ誤りがある場合など、検出できない共通原因が残る。

また、チェックポイント間で一時的に誤った表示が生じても、次の観測点までに状態が一致すれば、状態照合だけでは見逃す可能性がある。呼び出しの計測も手動であり、全呼び出しの網羅性を保証しない。卒研では、計測を削除・改変する試験を増やして、この限界を具体化する。

7.4 PASS・FAIL・UNKNOWN

PASS は、指定された有限記録で対象の条件を満たすことを表す。FAIL は不一致を表す。UNKNOWN は完了記録、必須チェックポイント、通知境界等が足りず、教材契約を判定できないことを表す。

契約判定器は `variant` という正常・比較のラベルを参照して合否を決めない。ラベルを付け替えても、実際の観測が同じなら同じ判定になることをテストする。実験スクリプトだけが、事前に定めた期待ラベルとの照合に用いる。

判定根拠はイベント番号として保持し、GUI の判定行から根拠へ移動できる。現在は判定を支える観測点を示す機能であり、原因を一意に特定したり、最小の反例を自動抽出したりする機能ではない。

8 GUI の操作と確認範囲

上部で教材、正常例／比較例、入力を指定する。中央にオブジェクト図、右側に教材説明・Java ソース・全記録の検証・正常例との比較を置き、下部に再生操作とイベント履歴を置く。線の色だけに依存せず、参照は実線、現在の呼び出しは橙の破線とラベルで区別する。

「全記録の検証」は再生位置にかかわらず記録全体を評価する。ソースの SHA-256 が一致するときだけ、イベントに対応する計測行を強調する。ソースが変わっている場合には誤った行対応を示さない。行は観測呼び出しの場所であり、Java の全命令を 1 行ずつステップ実行する意味ではない。

比較画面は、イベント番号ではなく `switch` や `notified` 等の操作名で対応する状態を比較する。現在は差のあるオブジェクト名を列挙する簡易方式であり、二画面の完全同期やフィールド単位の差分強調は未実装である。

画面は offscreen 環境で生成して確認し、日本語表示も確認した。自動操作試験では 4 教材 × 2variant の全再生位置、前進・後退、自動再生の切替、根拠への移動、ソース不一致時の表示を確認した。一般的なユーザビリティ評価は行っていない。

9 実施したパイロット評価

9.1 環境とコーパス

とりあえず、作ってみて検証してみた。検証環境は Python 3.12.14、PyQt6 6.11.0、Qt 6.11.0、OpenJDK 17.0.20、Linux x86_64 である。参照、Strategy、Observer は入力 0, 1, 100, 10000、State は固定シナリオを使用した。各教材に正常・比較の 2variant があるため、全 26 シナリオとなる。

教材	シナリオ数	イベント数	状態照合数	契約の所期判定
参照と同一性	8	12–13	16	8/8
Strategy	8	18	32	8/8
State	2	14–17	6	2/2
Observer	8	21–29	32	8/8
合計	26	—	86	26/26

イベント数は各記録の長さ、状態照合数は全シナリオを通じたチェックポイントの合計である。正常 13 件は PASS、比較 13 件は FAIL であった。観測整合性は正常例・比較例ともに全 86 点で一致した。

誤実装は 4 種類であり、比較 13 件は 13 種類の独立した誤りではない。また、開発時に利用した教材と同じ題材なので、未知実装に対する汎化精度とは解釈しない。100% という値を一般的な検出性能として宣伝しない。

9.2 自動試験

コア 13 試験と GUI4 試験、合計 17 試験が成功した。主な内容は次のとおりである。

- 同梱 8 記録の契約判定と状態照合。
- 正常・比較ラベルを入れ替えても判定が変わらないこと。
- 記録の途中切断や必須観測点の欠落が UNKNOWN になること。
- フィールド更新イベントを削除すると状態照合が FAIL になること。
- 保存・読込の一致、前進・後退の再構成一致、入力データを変更しないこと。
- 連番異常、未知参照、未対応スキーマ、壊れた JSON、終了後イベントの拒否。
- GUI の全教材・全 variant の操作、根拠への移動、ソースハッシュ不一致時の処理。

これは固定した回帰試験であり、任意の入力に対する形式的証明ではない。今後、境界条件と独立例を追加して改善する。

9.3 処理時間の予備測定

各記録についてウォームアップ後に 30 回、表示を伴わない状態再構成のみを測定した。記録ごとの中央値を教材内で集約した中央値は、参照 0.0112 ms、Strategy 0.0141 ms、State 0.01355 ms、Observer

0.0204 ms であった。小規模な記録のため、これらは大規模性能を示さない。GUI 応答時間、Java 起動時間、学生の作業時間とも異なる。

測定値は `results/pilot_live/scenarios.csv`、環境と制約は `summary.json`、Java 実行記録は同ディレクトリの `traces/` に保存した。Java 起動時間はソースのコンパイルを含み、単純な教材実行時間とは区別する。統計的な有意差検定は実施していない。

10 もし卒業研究としてやるとした場合に、追加すべき評価

10.1 独立した題材の追加

最優先は、同じパターンでも題材を変えた教材を少なくとも 1 例追加することである。例えば Strategy を料金計算から並べ替え方針へ、Observer を表示通知から在庫通知へ変更する。ただし、名前だけの置換ではなく、状態や入力構成を変える。

実装を変更する前に期待結果と契約を文章で定義する。既存判定器がそのまま使える部分、オブジェクト ID や契約の書き換えが必要な部分を分けて記録する。この過程で、現在の判定器が汎用ではないことを学生自身が確認できる。

10.2 誤実装と観測誤りを分ける

誤実装試験は、教材プログラムの要求違反を対象とする。観測誤り試験は、プログラム自体は正しいがイベントが不足・改変される場合を対象とする。両者を混在させない。

変異テストではプログラムに小さな変更を加えて検出能力を調べるが、同値変異や到達しない変異等の扱いが問題となる [8]。本パイロットは手作業で 4 種類の違反を埋め込んだ段階であり、汎用変異生成器ではない。卒研では、誤りの名称、変更箇所、期待される影響、観測条件を表にして追加する。

対象	追加候補	確認事項
Strategy	交換忘れ、誤った実体への委譲、数値だけの誤り	出力値だけで検出可能な誤りと、呼び出し観測が必要な誤りを分ける。
State	遷移先間違い、通過回数の更新忘れ	中間状態と最終結果の両方を調べる。
Observer	二重通知、解除忘れ、通知順序の交換	順序を要求しない契約で誤検出しないこと。
観測系	set 欠落、call 欠落、参照 ID 誤記、途中切断	FAIL と UNKNOWN を区別し、検出できない場合も記録する。

10.3 変更の局所性

変更課題をあらかじめ固定し、変更したファイル集合を M_t 、共通コアの集合を K とすると、課題 t におけるコア変更数を

$$c_t = |M_t \cap K|$$

で数えられる。これは理解時間ではなく、変更の広がりを示す構造的な指標である。共通コアの定義を結果に応じて変えない。

説明変更、役割追加、入力追加、新しい教材の追加、新しいイベントの追加を分けて測る。コードから設定ファイルへ移した作業も変更量に含める。対照となる別実装を大規模に作り直す前に、自分の変更履歴だけで何が言えるかを整理する。

11 今後の課題と検討事項

11.1 卒研内で優先する事項

1. **観測範囲の説明**。図に出る情報が実観測か、教材注釈か、契約からの解釈かを区別する。現在の宣言型と役割は注釈である。
2. **Java 理解の確認**。学生が、参照共有、多態性、State と Strategy の相違をコードに即して説明できるようにする。これは研究担当者の準備であり、被験者評価ではない。
3. **独立評価**。新しい題材、追加の誤り、計測漏れを用意する。成功例だけを増やさない。
4. **入力検証**。JSON スキーマの厳密化、チェックポイント内部の型検査、重複操作名、未対応の値の扱いを強化する。
5. **結果の可読な提示**。短い判定文、フィールドごとの差分、違反に関係する参照の強調を改善する。効果の実証とは区別する。

11.2 研究上の選択が必要な事項

教材追加方式 JSON だけで追加できる範囲をどこまで求めるか。過度な DSL 設計を避け、3 パターンの再利用可能な条件から始める。

実行計測方式 手動計測を維持するか、JDI 等を検討するか。卒研では手動方式の限界を測定することを優先し、汎用計測器は別段階とする。

基礎教材の範囲 今回の参照教材ではメソッド引数の値渡しや、static とインスタンス状態の違いは未実装である。パターン追加より先に基礎を補う価値があるか検討する。

比較の目的 正常対誤実装の比較と、パターン適用前後の設計比較は異なる。現パイロットは主に前者であり、後者は同じ要求を満たす二実装と変更課題が別途必要である。

保存と再生 長い記録を扱う必要が出た場合のみ、チェックポイントからの途中再開や状態キャッシュを追加する。

11.3 23 パターンへの展開

23 種類への対応は卒研の必須評価を終えた後の方向性である。優先度は、現在の観測で表せるか、新しい構造や意味が必要かで決める。

段階	対象	追加の論点
実装済み	Strategy、State、Observer	契約の一般化と独立題材。
比較的近い拡張	Factory Method、Abstract Factory、Template Method、Adapter、Facade、Proxy、Bridge、Mediator	生成・委譲の役割、静的構造の注釈。
構造の拡張	Composite、Decorator、Chain of Responsibility、Visitor、Interpreter	再帰、ネストした呼び出し、二重ディスパッチ等。

段階	対象	追加の論点
状態・同一性の 拡張	Builder、Prototype、Singleton、Flyweight、 Command、Iterator、Memento	構築途中、複製、共有範囲、 履歴、復元。

この区分は実装工数の保証ではない。Singleton の並行実行、Prototype の深いコピー、Command の Undo などは追加条件が必要であり、すべてを基本パターンの必須性質にしない。

12 たとえば、12 週間の実施例と判断基準

もし、卒業研究としてこのテーマをやるとして、担当学生が Python と Java のクラスを一通り学んでいることを仮定し、週 8 時間、12 週間で 96 時間を基本枠として考えるとこんな感じのスケジュールにやるんじゃないかな。

週	作業	確認する成果
1-2	パイロット実行、Java と Python の流れ、文献読解	1 教材の処理と観測境界を説明する。
3-4	契約・観測仕様の明文化、独立題材の設計	期待結果と追加すべき観測を先に書く。
5-6	独立題材、追加誤実装、回帰試験	正常・異常・観測不足を再現する。
7-8	欠損試験、変更箇所記録、必要な修正	検出できた範囲と見逃した範囲の表。
9-10	条件固定の評価、結果分析	実験を一括再実行し結果表を作る。
11-12	卒論、発表、再現手順、限界の整理	実装原理と研究結果を区別して説明する。

第 4 週で観測仕様が固まらなければ新パターン追加を中止する。第 8 週で新機能を凍結し、評価と執筆に移る。予定どおり進んでも 23 種類への拡張を卒研期間内に義務化しない。

13 提供物と学生自身の研究成果の境界

卒研として学生が担う具体的な作業は、観測仕様の精査、独立題材の作成、誤実装・欠損試験の追加、変更の波及分析、結果の解釈である。新機能が少なくても、既存の仕組みの限界を再現し、原因と改善案を示せば研究の中心になる。提供コードの量が多い場合は、最初に 1 教材へ絞って理解してから拡張する。

14 妥当性と未実装事項

本結果は、同梱の単一スレッド教材、限定した整数入力、手動計測、固定したオブジェクト ID と操作名に依存する。一般 Java の意味保存、任意コードからのパターン認識、全 Java 型への対応、並行実行、GC、例外経路は評価していない。

クラス構造は文章による注釈表示であり、UML 自動生成ではない。条件分岐実装とパターン実装の保守性比較、変更影響の自動計算、汎用契約 DSL、Java コード編集 IDE、最小反例抽出も未実装である。実オブジェクトの最終結果と状態を見ても、計測前後で振る舞いが変わらないことを一般的に保証する試験はまだない。

評価データは小さく、同じ誤実装の複数入力を含むため、独立標本として有意差を計算しない。未知題材での失敗も含めた全件表を優先する。人間の学習効果や理解容易性は本研究の評価外である。

15 まとめ

卒業研究としてやるのであれば、少数の Java 教材を忠実に可視化し、正しく表示できたかと要求を満たすかを分けて検証する計画が現実的である。Python・PyQt6 の採用は、教材・表示・評価を整理するための実装条件として妥当であるが、それ自体を新規性や可読性向上の証拠とはしない。

パイロットは 4 教材、正常・比較 8 記録、Java 再実行、状態照合、根拠への移動、自動評価を実装し、限定したコーパスで動作を確認した。今後は機能数を増やす前に、独立題材と観測漏れ試験を追加し、適用できる範囲と限界を学生自身が示すことを優先する。

付録 A 起動と評価の手順

配布 ZIP を展開し、そのディレクトリで実行する。Python 3.10 以降を想定する。

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pattern_lab
```

同梱記録の再生には JDK は不要である。Java 再実行には JDK 17 以降を用意する。追加 Java ライブラリは不要である。

```
python -m unittest discover -s tests -v
python evaluate.py --out results/my_replay
python evaluate.py --live --out results/my_live
python -m pattern_lab --regenerate
```

PyQt6 がない環境でもコアの評価は可能であり、GUI 試験はスキップされる。JDK がない場合はライブ評価が失敗するため、記録再生による評価と区別する。詳細は同梱 README と docs/TRACE_FORMAT.md を参照する。

参考文献

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. ISBN 978-0-201-63361-0. <https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610>.
- [2] Oracle, *The Java Language Specification, Java SE 21 Edition*, Chapter 4, Types, Values, and Variables, especially Sections 4.3.1 and 4.12.2. <https://docs.oracle.com/javase/specs/jls/se21/html/jls-4.html>.
- [3] Oracle, *The Java Language Specification, Java SE 17 Edition*, Section 15.12.4.4, Locate Method to Invoke. <https://docs.oracle.com/javase/specs/jls/se17/html/jls-15.html>.
- [4] A. Moreno, N. Myller, E. Sutinen, and M. Ben-Ari, “Visualizing programs with Jeliot 3,” *Proceedings of the Working Conference on Advanced Visual Interfaces*, 2004, pp. 373–376. DOI: <https://doi.org/10.1145/989863.989928>.
- [5] M. Weninger, S. Grünbacher, and H. Prähofer, “JavaWiz: A Trace-Based Graphical Debugger for Software Development Education,” *Proceedings of ICPC 2025*, pp. 1–12, 2025. DOI: <https://doi.org/10.1109/ICPC66645.2025.00023>. Project and publication information: <https://javawiz.net/>.
- [6] M. Weninger, “Towards Guided Omniscient Debugging in Education Using Pedagogical Execution Traces (PETs),” *Fourth Workshop on Future Debugging Techniques (DEBT)*, 2026, pp. 1–4. Publication information and abstract: <https://javawiz.net/>.
- [7] D. Pritchard and contributors, `java_visualize`: Visualization for Java using the On-line Python Tutor frontend, repository README. https://github.com/daveagp/java_visualize.
- [8] Y. Jia and M. Harman, “An Analysis and Survey of the Development of Mutation Testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011. DOI: <https://doi.org/10.1109/TSE.2010.62>.
- [9] Riverbank Computing, *PyQt6 Reference Guide*. <https://www.riverbankcomputing.com/static/Docs/PyQt6/>.
- [10] The Qt Company, *Qt 6 Documentation: QGraphicsScene*. <https://doc.qt.io/qt-6/qgraphicsscene.html>.
- [11] Oracle, *Java Platform, Standard Edition 17: The java Command*, Using Source-File Mode to Launch Single-File Source-Code Programs. <https://docs.oracle.com/en/java/javase/17/docs/specs/man/java.html>.

Web 資料の確認日：2026 年 10 月 4 日。文献の機能記述と本実装の実測結果は区別して記載した。